

MLOps (S2-25_AIMLCZG523) Assignment 2

Binary image classification (Cats vs Dogs) for a pet adoption platform

End-to-End MLOps Implementation

Name: RAJ KUMAR M

BITS ID: 2024AD05110

Email: 2024AD05110@wilp.bits-pilani.ac.in

Course: MTech AI & ML

GitHub Repository link:

<https://github.com/RAJKUMAR27M/cats-dogs-mlops-assignment2.git>

Final Demo video location link:

https://drive.google.com/file/d/1_U2VYAa3dZED1Pxb_FuAlr3QS98JG5u/view?usp=drive_link

Pipeline Architecture Overview

This project implements a complete MLOps pipeline covering model development, experiment tracking (MLflow), packaging (FastAPI + Docker), CI/CD (GitHub Actions), deployment (Docker Compose / Kubernetes), and monitoring (Prometheus).

Project Structure

```
cats-dogs-mlops/
├── .dvc/                  # DVC configuration
├── .github/workflows/     # CI (ci.yml) and CD (cd.yml) pipelines
├── api/                   # FastAPI inference service
│   ├── main.py            # Endpoints: /health /predict /stats /metrics
│   └── predictor.py        # Model loading & inference
├── k8s/                   # Kubernetes manifests
├── monitoring/            # Prometheus configuration
├── scripts/               # Utility scripts
│   ├── download_data.py   # Kaggle dataset download
│   ├── preprocess_data.py # DVC stage helper
│   ├── smoke_test.py      # Post-deploy smoke tests
│   └── batch_evaluate.py  # M5 performance tracking
├── src/
│   ├── data/preprocess.py # Image loading, normalisation, split logic
│   └── models/
│       ├── cnn_model.py   # 4-block CNN architecture
│       └── train.py        # Training with MLflow tracking
├── tests/                 # pytest unit tests
├── Dockerfile
├── docker-compose.yml     # API + Prometheus
├── dvc.yaml               # DVC pipeline stages
├── params.yaml            # Hyperparameters
└── requirements.txt
```



Figure 1: End-to-end MLOps pipeline architecture

M1: Model Development & Experiment Tracking

Build a baseline CNN, track experiments with MLflow, version data/code with Git + DVC.

Data & Code Versioning (Git + DVC)

Git tracks all source code; DVC tracks large data and model artifacts.

Git initialisation

```
git init
git add .
git commit -m "Initial project structure"
git remote add origin https://github.com/RAJKUMAR27M/cats-dogs-mlops-assignment2.git
git push -u origin main
```

DVC setup

```
pip install dvc
dvc init
dvc remote add -d local_remote ../dvc_store
git add .dvc .dvcignore
git commit -m "Initialise DVC"
dvc push
```

Download dataset

Manual download (recommended):

- Go to <https://www.kaggle.com/datasets/bhavikjikadara/dog-and-cat-classification-dataset>
- Click the Download button (downloads dog-and-cat-classification-dataset.zip)
- Extract the zip — you should get Cat\ and Dog\ folders
- Place them under data\raw\ so the structure is:
 - data\raw\Cat*.jpg
 - data\raw\Dog*.jpg
- Skip the download script below — go directly to Step 4 (preprocess)

```
dvc add data/raw
git add data/raw.dvc
git commit -m "Track raw data with DVC"
```

Only needed if using Kaggle API instead of manual download:

```
python scripts/download_data.py --output-dir data/raw
```

Model Building (Custom CNN)

A 4-block CNN with BatchNorm, Dropout, and GlobalAveragePooling is implemented. Input: 224x224 RGB. Output: sigmoid (0 = cat, 1 = dog). Model saved as model.pt.

Training commands

- With real dataset
`python src/models/train.py --data-dir data/processed --artifacts-dir artifacts`
- Full DVC pipeline (preprocess + train)
`dvc repro`
- Dry-run (synthetic data, no Kaggle needed)
`python src/models/train.py --dry-run`

Task 3 – Experiment Tracking with MLflow

MLflow logs: hyperparameters, per-epoch accuracy/loss, test accuracy, confusion matrix PNG, training curves PNG, classification report, and model artifact.

MLflow commands

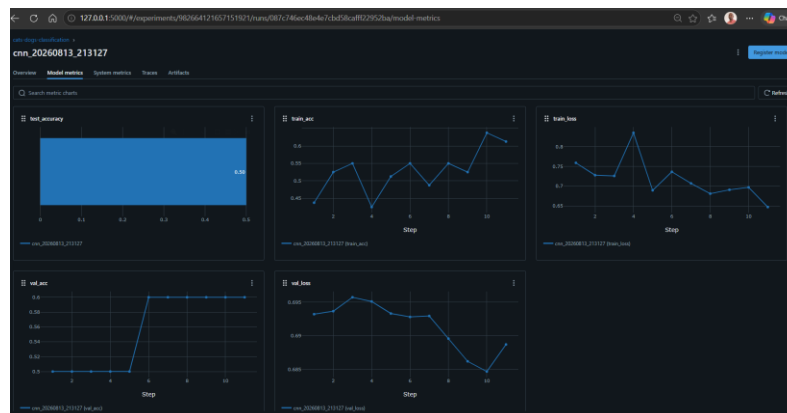
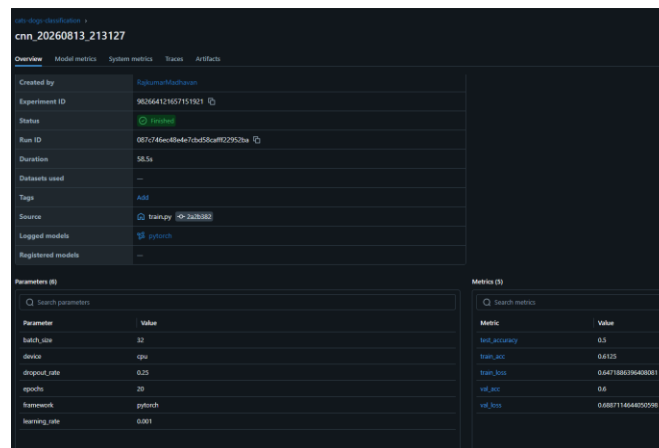
Launch MLflow UI

`mlflow ui --host 127.0.0.1 --port 5000`

→ `http://localhost:5000`

`mlflow experiments list`

`mlflow runs list --experiment-name cats-dogs-classification`



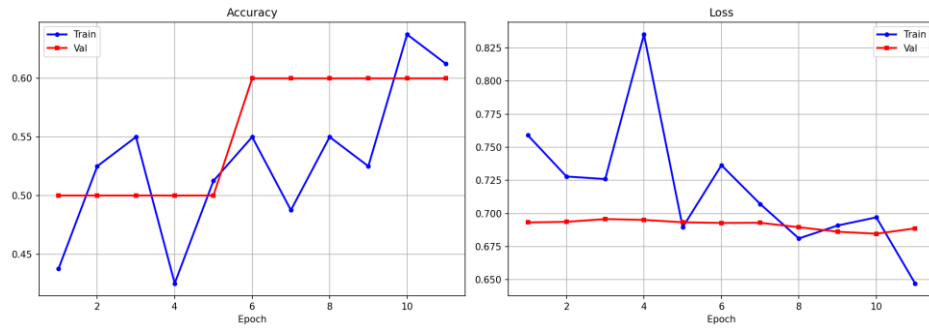


Figure 2: Training and validation accuracy/loss curves

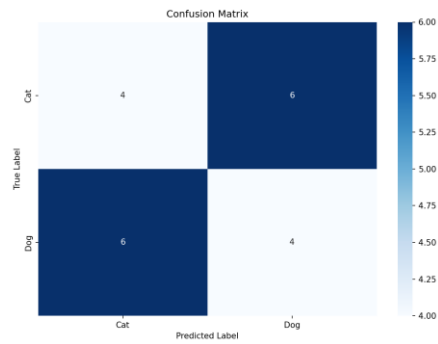


Figure 3: Confusion matrix on the evaluation split

classification_report.txt 334B

Path: file:///C:/Users/RajkumarMadhavan/Downloads/submission

	precision	recall	f1-score	support
cat	0.00	0.00	0.00	5
dog	0.50	1.00	0.67	5
accuracy			0.50	10
macro avg	0.25	0.50	0.33	10
weighted avg	0.25	0.50	0.33	10

Latest recorded metrics

Metric	Value
test accuracy	0.5
train accuracy	0.6125
val accuracy	0.6

Logged MLflow parameters

Parameter	Value
model_type	custom_cnn
input_size	224×224
batch_size	32
epochs	20
learning_rate	0.001
dropout_rate	0.25
optimizer	Adam
loss	BCELoss

M2: Model Packaging & Containerization

Wrap the model as a reproducible Docker-based REST API.

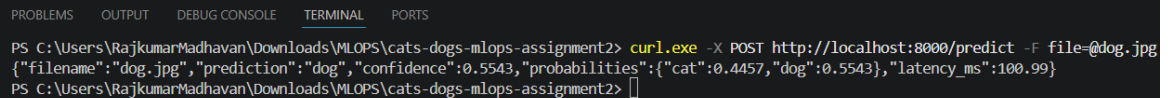
FastAPI Inference Service

Endpoints:

- GET /health – service status + model_loaded flag
- POST /predict – upload image → label, confidence, probabilities, latency_ms
- GET /stats – request count and average latency
- GET /metrics – Prometheus metrics

curl test commands

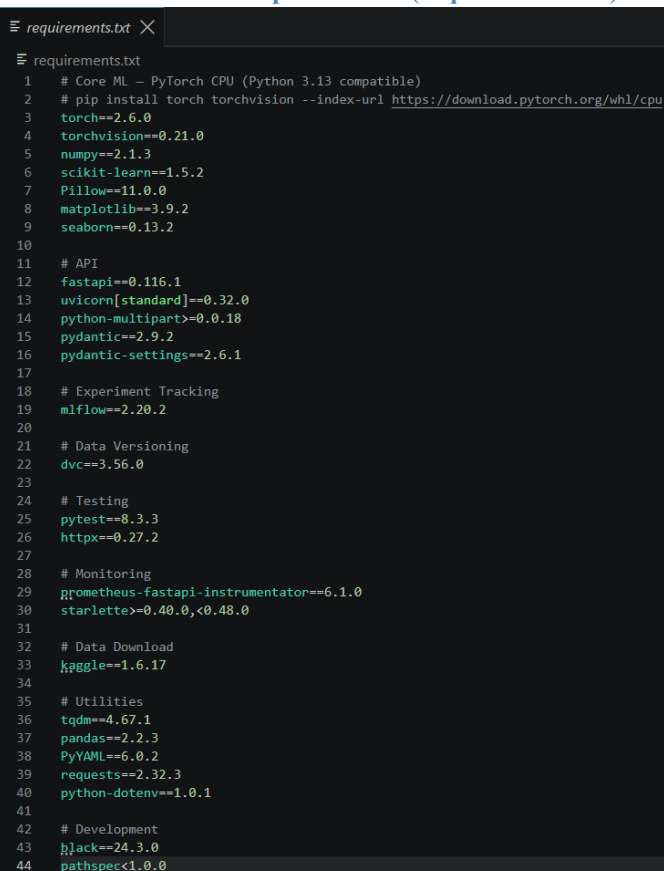
- Health check
curl <http://localhost:8000/health>
- Predict
curl.exe -X POST http://localhost:8000/predict -F [file=@dog.jpg](#)
- Expected response:



```
PS C:\Users\RajkumarMadhavan\Downloads\MLOPS\cats-dogs-mlops-assignment2> curl.exe -X POST http://localhost:8000/predict -F file=@dog.jpg
{"filename":"dog.jpg","prediction":"dog","confidence":0.5543,"probabilities":{"cat":0.4457,"dog":0.5543},"latency_ms":100.99}
PS C:\Users\RajkumarMadhavan\Downloads\MLOPS\cats-dogs-mlops-assignment2>
```

- Prometheus metrics
curl.exe <http://localhost:8000/metrics>

Task 2 – Environment Specification (requirements.txt)



```
requirements.txt
# Core ML – PyTorch CPU (Python 3.13 compatible)
# pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
torch==2.6.0
torchvision==0.21.0
numpy==2.1.3
scikit-learn==1.5.2
Pillow==11.0.0
matplotlib==3.9.2
seaborn==0.13.2

# API
fastapi==0.116.1
uvicorn[standard]==0.32.0
python-multipart==0.0.18
pydantic==2.9.2
pydantic-settings==2.6.1

# Experiment Tracking
mlflow==2.20.2

# Data Versioning
dvc==3.56.0

# Testing
pytest==8.3.3
httpx==0.27.2

# Monitoring
prometheus-fastapi-instrumentator==6.1.0
starlette==0.40.0,<0.48.0

# Data Download
kaggle==1.6.17

# Utilities
tqdm==4.67.1
pandas==2.2.3
PyYAML==6.0.2
requests==2.32.3
python-dotenv==1.0.1

# Development
black==24.3.0
pre-commit==3.7.1
```

- System Python (preferred — no venv needed)

pip install -r requirements.txt

- Optional: virtual environment (create it INSIDE cats-dogs-mlops)

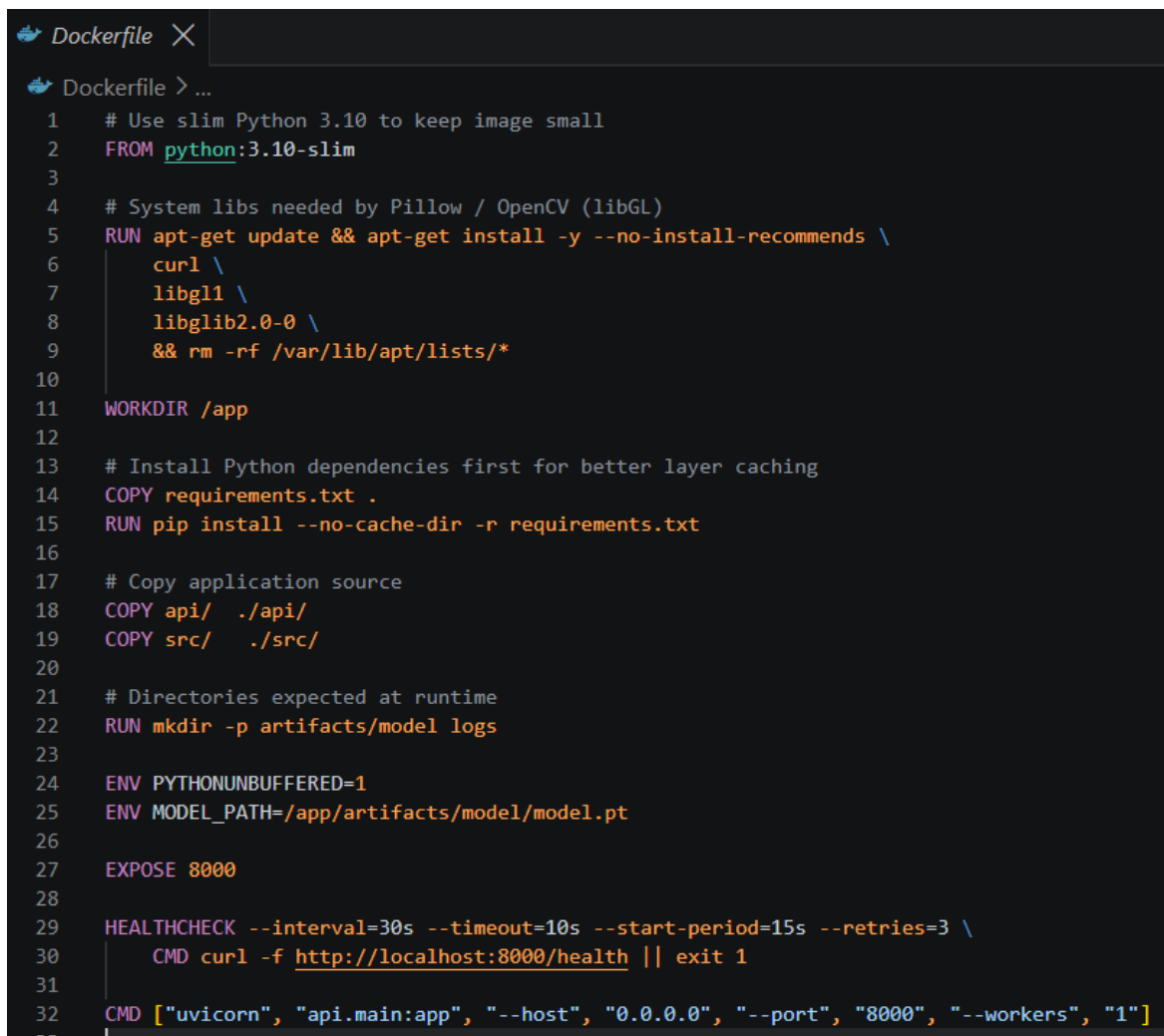
python -m venv .venv

.venv\Scripts\activate

pip install -r requirements.txt

Containerisation (Docker)

Dockerfile

A screenshot of a code editor showing a Dockerfile. The editor has a dark theme. The Dockerfile content is as follows:

```
1 # Use slim Python 3.10 to keep image small
2 FROM python:3.10-slim
3
4 # System libs needed by Pillow / OpenCV (libGL)
5 RUN apt-get update && apt-get install -y --no-install-recommends \
6     curl \
7     libgl1 \
8     libglib2.0-0 \
9     && rm -rf /var/lib/apt/lists/*
10
11 WORKDIR /app
12
13 # Install Python dependencies first for better layer caching
14 COPY requirements.txt .
15 RUN pip install --no-cache-dir -r requirements.txt
16
17 # Copy application source
18 COPY api/ ./api/
19 COPY src/ ./src/
20
21 # Directories expected at runtime
22 RUN mkdir -p artifacts/model logs
23
24 ENV PYTHONUNBUFFERED=1
25 ENV MODEL_PATH=/app/artifacts/model/model.pt
26
27 EXPOSE 8000
28
29 HEALTHCHECK --interval=30s --timeout=10s --start-period=15s --retries=3 \
30     CMD curl -f http://localhost:8000/health || exit 1
31
32 CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "1"]
33
```

Docker build & run

- Build
docker build -t cats-dogs-api:latest .

- Deploy with Docker Compose

```
# Start API + Prometheus
docker rm -f cats-dogs-api
docker container prune
    ### Type y when prompted to confirm
docker compose up -d

# Check status
docker compose ps
docker compose logs -f api
```

- Verify
curl <http://localhost:8000/health>
- Stop
docker compose down

M3: CI Pipeline – Build, Test & Image Creation

Automated Testing with pytest

Unit tests cover:

- Preprocessing: load_and_resize_image, normalize_image, preprocess_image
- Inference: Predictor class (label, confidence, edge cases)
- API: /health endpoint structure

Run tests

pytest tests/ -v

With coverage report

pytest tests/ -v --cov=src --cov=api --cov-report=html

```
PS C:\Users\RajkumarMadhavan\Downloads\VL OPS\cats-dogs-mlops-assignment2> pytest tests/ -v --cov=src --cov=api --cov-report=html
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.3, pluggy-1.6.0 -- C:\Python\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\RajkumarMadhavan\Downloads\VL OPS\cats-dogs-mlops-assignment2
configfile: pytest.ini
plugins: anyio-4.9.0, hydra-core-1.3.4, cov-5.0.0
collected 23 items

tests/test_inference.py::TestPredictor::test_returns_label PASSED [ 4%]
tests/test_inference.py::TestPredictor::test_returns_confidence_in_range PASSED [ 8%]
tests/test_inference.py::TestPredictor::test_returns_raw_score_in_range PASSED [ 13%]
tests/test_inference.py::TestPredictor::test_high_score_predicts_dog PASSED [ 17%]
tests/test_inference.py::TestPredictor::test_low_score_predicts_cat PASSED [ 21%]
tests/test_inference.py::TestPredictor::test_is_loaded_true PASSED [ 26%]
tests/test_inference.py::TestPredictor::test_is_loaded_false_when_no_model PASSED [ 30%]
tests/test_inference.py::TestPredictor::test_preprocess_output_shape PASSED [ 34%]
tests/test_inference.py::TestPredictor::test_preprocess_values_normalized PASSED [ 39%]
tests/test_inference.py::TestPredictor::test_predict_raises_without_model PASSED [ 43%]
tests/test_inference.py::TestHealthEndpoint::test_health_returns_200 PASSED [ 47%]
tests/test_inference.py::TestHealthEndpoint::test_health_has_required_fields PASSED [ 52%]
tests/test_inference.py::TestHealthEndpoint::test_health_status_value PASSED [ 56%]
tests/test_preprocess.py::TestLoadAndResizeImage::test_resizes_to_224x224 PASSED [ 60%]
tests/test_preprocess.py::TestLoadAndResizeImage::test_output_is_rgb PASSED [ 65%]
tests/test_preprocess.py::TestLoadAndResizeImage::test_custom_target_size PASSED [ 69%]
tests/test_preprocess.py::TestLoadAndResizeImage::test_grayscale_converted_to_rgb PASSED [ 73%]
tests/test_preprocess.py::TestNormalizeImage::test_values_in_unit_range PASSED [ 76%]
tests/test_preprocess.py::TestNormalizeImage::test_output_dtype_Float32 PASSED [ 82%]
tests/test_preprocess.py::TestNormalizeImage::test_known_values PASSED [ 86%]
tests/test_preprocess.py::TestNormalizeImage::test_shape_preserved PASSED [ 91%]
tests/test_preprocess.py::TestPreprocessImage::test_output_shape PASSED [ 95%]
tests/test_preprocess.py::TestPreprocessImage::test_values_normalized PASSED [ 100%]

----- coverage: platform win32, python 3.11.9-final-0 -----
Coverage HTML written to dir htmlcov

===== 23 passed in 2.87s =====
PS C:\Users\RajkumarMadhavan\Downloads\VL OPS\cats-dogs-mlops-assignment2>
```

CI with GitHub Actions

CI pipeline steps

1. Checkout repository
2. Set up Python 3.10 with pip cache
3. pip install -r requirements.txt
4. pytest tests/ -v
5. docker/setup-buildx
6. docker/login-action (Docker Hub)
7. docker/build-push-action (push only on main)

Artifact Publishing (Docker Hub)

On every successful push to main the CI workflow tags the image as:

```
<username>/cats-dogs-api:latest
<username>/cats-dogs-api:main
<username>/cats-dogs-api:main-<sha>
```

Manual Docker Hub push

```
docker login -u <username>
docker tag cats-dogs-api:latest <username>/cats-dogs-api:latest
docker push <username>/cats-dogs-api:latest
```

Pull on another machine

```
docker pull <username>/cats-dogs-api:latest
```

M4: CD Pipeline & Deployment

Deployment commands

- Deploy
docker compose up -d
- Check status
docker compose ps
docker compose logs -f api
- Rolling update
docker compose pull && docker compose up -d --remove-orphans
- Kubernetes (optional)
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl rollout status deployment/cats-dogs-api

CD / GitOps with GitHub Actions

Smoke Tests

After deployment the CD pipeline runs scripts/smoke_test.py which verifies:

1. /health returns HTTP 200 with status='healthy'
2. /predict accepts an image and returns a valid prediction

```
PS C:\Users\RajkumarMadhavan\Downloads\MILOPS\cats-dogs-mlops-assignment2> python scripts/smoke_test.py --base-url http://localhost:8000
2026-08-15 15:23:54,561 - INFO - === Smoke tests against http://localhost:8000 ===
2026-08-15 15:23:54,868 - INFO - Service ready after 1 attempt(s)
2026-08-15 15:23:54,868 - INFO - → Testing /health ...
2026-08-15 15:23:54,885 - INFO - PASS: {'status': 'healthy', 'model_loaded': True, 'version': '1.0.0', 'request_count': 13, 'avg_latency_ms': 35.07}
2026-08-15 15:23:54,903 - INFO - → Testing /predict ...
2026-08-15 15:23:55,000 - INFO - PASS: prediction=dog confidence=0.5372 latency_ms=60.8
2026-08-15 15:23:55,000 - INFO - === Results ===
2026-08-15 15:23:55,000 - INFO - health PASSED
2026-08-15 15:23:55,000 - INFO - predict PASSED
2026-08-15 15:23:55,000 - INFO - All smoke tests PASSED ✓
PS C:\Users\RajkumarMadhavan\Downloads\MILOPS\cats-dogs-mlops-assignment2>
```

Pipeline fails and rolls back if any smoke test fails.

Post-deploy smoke tests – health check and prediction endpoint (M4)

Run smoke tests manually

- python scripts/smoke_test.py --base-url <http://localhost:8000>
- Expected output:

```
PS C:\Users\RajkumarMadhavan\Downloads\MILOPS\cats-dogs-mlops-assignment2> python scripts/smoke_test.py --base-url http://localhost:8000
2026-08-15 15:23:54,561 - INFO - === Smoke tests against http://localhost:8000 ===
2026-08-15 15:23:54,868 - INFO - Service ready after 1 attempt(s)
2026-08-15 15:23:54,868 - INFO - → Testing /health ...
2026-08-15 15:23:54,885 - INFO - PASS: {'status': 'healthy', 'model_loaded': True, 'version': '1.0.0', 'request_count': 13, 'avg_latency_ms': 35.07}
2026-08-15 15:23:54,903 - INFO - → Testing /predict ...
2026-08-15 15:23:55,000 - INFO - PASS: prediction=dog confidence=0.5372 latency_ms=60.8
2026-08-15 15:23:55,000 - INFO - === Results ===
2026-08-15 15:23:55,000 - INFO - health PASSED
2026-08-15 15:23:55,000 - INFO - predict PASSED
2026-08-15 15:23:55,000 - INFO - All smoke tests PASSED ✓
PS C:\Users\RajkumarMadhavan\Downloads\MILOPS\cats-dogs-mlops-assignment2>
```

M5: Monitoring, Logs & Final Submission

Request/Response Logging & Metrics

The FastAPI service implements:

- Python logging (api.requests logger) for every prediction (filename, pred, confidence, latency)
- prometheus-fastapi-instrumentator exposing /metrics for Prometheus scraping

- In-app counters: request_count, total_latency
- /stats endpoint for quick dashboard

Prometheus configuration

global:

scrape_interval: 15s
evaluation_interval: 15s

scrape_configs:

- job_name: cats-dogs-api
 - static_configs:
 - targets: ['api:8000']
 - metrics_path: /metrics
 - scrape_interval: 10s
- job_name: prometheus
 - static_configs:
 - targets: ['localhost:9090']

Start monitoring stack

docker compose up -d # starts API + Prometheus
Prometheus UI → http://localhost:9090

Useful PromQL queries:

http_requests_total
http_request_duration_seconds_bucket
rate(http_requests_total[5m])

curl http://localhost:8000/metrics # raw Prometheus metrics
curl http://localhost:8000/stats # JSON summary

Sample log output

```
2024-11-01 10:23:45 - api.requests - INFO - PREDICT | file=cat.jpg | pred=cat | conf=0.9234 |
latency=42.5ms
2024-11-01 10:23:46 - api.requests - INFO - PREDICT | file=dog.jpg | pred=dog | conf=0.8891 |
latency=38.2ms
2024-11-01 10:24:10 - api.requests - INFO - PREDICT | file=test.png | pred=cat | conf=0.7120 |
latency=51.0ms
```

Task 2 – Post-Deployment Performance Tracking

scripts/batch_evaluate.py sends a batch of images (real or synthetic) to the running API, computes accuracy / precision / recall / F1 / P95-latency, and logs them to MLflow.

Run batch evaluation

```
python scripts/batch_evaluate.py \
--base-url http://localhost:8000 \
--data-dir data/processed \
--output-dir artifacts
```

Writes artifacts/post_deployment_metrics.json

Example output:

```
# {"num_samples":100,"accuracy":0.87,"precision":0.88,
# "recall":0.86,"f1_score":0.87,"avg_latency_ms":45.2,"p95_latency_ms":78.3}
```

Complete End-to-End Workflow

Step 1 – Install dependencies

```
python -m pip install -r requirements.txt
```

Step 2 – Init Git & DVC

```
git init && git add . && git commit -m 'init'  
dvc init && dvc remote add -d local_remote ../dvc_store
```

Step 3 – Download dataset

```
set KAGGLE_USERNAME=xxx && set KAGGLE_KEY=yyy  
python scripts/download_data.py --output-dir data/raw
```

Step 4 – Preprocess & Train (DVC)

```
dvc repro
```

Step 5 – View MLflow UI

```
mlflow ui --host 0.0.0.0 --port 5000
```

Step 6 – Run tests

```
pytest tests/ -v
```

Step 7 – Build Docker image

```
docker build -t cats-dogs-api:latest .
```

Step 8 – Deploy

```
docker compose up -d
```

Step 9 – Smoke tests

```
python scripts/smoke_test.py --base-url http://localhost:8000
```

Step 10 – Monitor

```
Prometheus: http://localhost:9090  
curl http://localhost:8000/metrics
```

Step 11 – Batch evaluation

```
python scripts/batch_evaluate.py --base-url http://localhost:8000
```

Conclusion

M1: A custom 4-block CNN with BatchNorm and Dropout is trained on 224x224 RGB images. All runs are tracked with MLflow; dataset and model are versioned with DVC.

M2: The trained model is served via FastAPI with /health and /predict endpoints, containerised with Docker, and deployed reproducibly with Docker Compose.

M3: GitHub Actions CI automatically runs pytest, builds the Docker image, and pushes it to Docker Hub on every commit to main.

M4: The CD workflow pulls the new image, deploys with Compose, and runs smoke tests to validate each release. Kubernetes manifests are also provided.

M5: prometheus-fastapi-instrumentator exposes Prometheus metrics; every prediction is logged with filename, class, confidence, and latency. Post-deployment batch evaluation tracks drift over time and logs results to MLflow. The GitHub repository URL and published container image reference should be included in the final submission report alongside the zip archive.

GitHub Repository link:

<https://github.com/RAJKUMAR27M/cats-dogs-mlops-assignment2.git>

Final Demo video location link:

https://drive.google.com/file/d/1_U2VYAa3dZED1Pxb_FuAlr3QS98ICg5u/view?usp=drive_link